

LAB 10

POST LAB TASKS

Task 1:

A university has developed a secure digital library system where multiple students and faculty members can access research documents simultaneously. In this system, users are categorized into two types: readers (students who only view documents) and writers (faculty members who update or upload new research material). The system must ensure that multiple readers can access the database concurrently without affecting each other, but when a writer is updating the database, no reader should be allowed to access it, and no other writer should interfere during the update process.

Additionally, to maintain fairness, the system should avoid starvation, meaning writers should not be indefinitely delayed by continuous reader access. The administrator has asked you to design a synchronization solution using mutex locks and semaphores to manage concurrent access to the shared database. Your implementation should ensure data consistency, prevent race conditions, and allow efficient utilization of system resources.

You are required to simulate this system using threads where multiple reader and writer threads attempt to access the shared resource. Demonstrate how your solution handles concurrent reads, exclusive writes, and fairness constraints. Also, analyze the behavior of your system under heavy load (e.g., many readers and few writers) and explain how synchronization primitives maintain correctness.

Task 2:

A food delivery company operates a real-time order processing system where customer orders are continuously received and processed by a team of delivery agents. The system consists of a fixed-size order queue (buffer) that temporarily stores incoming orders before they are assigned to delivery agents. Here, producer threads represent customers placing orders, while consumer threads represent delivery agents picking up orders for delivery.

Since the buffer has limited capacity, the system must ensure that producers do not add new orders when the queue is full, and consumers should not attempt to retrieve orders when the queue is empty. To maintain system stability and avoid data inconsistency, proper synchronization is required. The company also wants to ensure that order processing is efficient and no thread remains blocked unnecessarily.

You are tasked with implementing this system using mutexes for mutual exclusion and counting semaphores to keep track of available slots and filled slots in the buffer. Your program should simulate multiple producers and consumers working concurrently, handling real-time constraints such as varying order arrival rates and delivery processing times.

Additionally, extend your solution to handle edge cases such as sudden spikes in order requests or delays in delivery processing. Analyze how your synchronization design prevents deadlocks, race conditions, and buffer overflow/underflow, and discuss the role of semaphores in coordinating thread execution.